Imię i Nazwisko Kacper Zych	Kierunek Informatyka Techniczna	Rok studiów i grupa 2026/2027
Data zajęć 04.11.2026	Numer i temat sprawozdania 6. Logika i tautologie	

Zajecia polegały na poznaniu pojec tautologi i logiki, poznania podstawowych zdań logicznych oraz opracowanie tzw. tablicy prawdy dla kazdego przypadku q i p z każdym podstawowym zdaniem logicznym a także wytlumaczenie zdan logicznych w języku python przy uzyciu operatorow ==, is, not, not is, or, and.

Zadanie 1:

```
def negacja(a):
    return not a

def koniunkcja(a, b):
    return (a and b)

def alternatywa(a, b):
    return (a or b)

def implikacja(a,b):
    if a == True and b == False:
        return False
    else:
        return True

def rownowaznosc(a,b):
    if a == b:
        return True
    else:
        return False

poprawne = True

a = input("Podaj wartosc a (0, 1): ")
match a:
    case "0":
        a = False
    case "1":
        a = True
    case _:
        print("to nie jest 0 albo 1")
        poprawne = False
b = input("Podaj wartosc b (0, 1): ")



match b:
    case "0":
        b = False
```

```
    case "1":
        b = True
    case _:
        print("to nie jest 0 albo 1")
        poprawne = False

if poprawne:
    print("negacja: ", negacja(a), negacja(b))
    print("koniunkcja: ", koniunkcja(a,b))
    print("alternatywa: ", alternatywa(a, b))
    print("implikacja: ", implikacja(a, b))
    print("równoważnosc: ", rownowaznosc(a, b))
```

```
Podaj wartosc a (0, 1): 1
Podaj wartosc b (0, 1): 0
negacja:  False True
koniunkcja:  False
alternatywa:  True
implikacja:  False
równoważnosc:  False
```

Zadanie 2:

```
p = True
q = True
r = False

print("1", koniunkcja(p, q))
print("2", alternatywa(p, q))
print("3", koniunkcja(negacja(p), alternatywa(p, q)))
print("4", implikacja(koniunkcja(p,r),q))
print("5", negacja(rownowaznosc(p, alternatywa(q, r))))
print("6", rownowaznosc(alternatywa(implikacja(p,r),
negacja(q)),implikacja(p,koniunkcja(r, negacja(q)))))
print("7",
implikacja(alternatywa(alternatywa(negacja(r),q),negacja(koniunkcja(q,r))),
negacja(implikacja(q,p))))
```

```
1 True
2 True
3 False
4 True
5 False
6 True
7 False
```

p	q	r	p i q	p lub q	nie p	nie p i (p lub q)	(p i r) wtedy q	nie (p <-> (q lub r))	[(p -> r) lub nie q] <-> [p -> (r i nie q)]	[(nie r lub q) lub nie (q i r)] -> [nie(q ->p)]
1	1	0	1	1	0	0	1	0	[0] <-> [0]	[(1 lub 1)] -> [0]
									1	0

Zadanie 3:

```
lista = [0,1,2,3,4,56,7,8,9,19]
a = int(input("Podaj liczbe calkowita: "))
if a in lista:
    print("Nalezy do listy")
if a not in lista:
    print("nie nalezy do listy")
```

```
Podaj liczbe calkowita: 4
Nalezy do listy
```

```
Podaj liczbe calkowita: 6
nie nalezy do listy
```

Zadanie 4:

```
koszyk1 = ["chleb", "mleko", "jajka"]
koszyk2 = koszyk1
```

```
koszyk3 = ["chleb", "mleko", "jajka"]

print("koszyk1 == koszyk2", koszyk1 == koszyk2)
print("koszyk1 == koszyk3", koszyk1 == koszyk3)
print("koszyk2 == koszyk3", koszyk2 == koszyk3)
print("koszyk1 is koszyk2", koszyk1 is koszyk2)
print("koszyk1 is koszyk3", koszyk1 is koszyk3)
print("koszyk2 is koszyk3", koszyk2 is koszyk3)

koszyk2.append("woda")

print(koszyk1, koszyk2, koszyk3)
```

```
koszyk1 == koszyk2 True
koszyk1 == koszyk3 True
koszyk2 == koszyk3 True
koszyk1 is koszyk2 True
koszyk1 is koszyk3 False
koszyk2 is koszyk3 False
['chleb', 'mleko', 'jajka', 'woda'] ['chleb', 'mleko', 'jajka', 'woda'] ['chleb', 'mleko', 'jajka']
```

znak „==” porównuje poszczególne wartości podanych wyrazów a „is” sprawdza czy są w pamięci tym samym. koszyk 1 i koszyk 2 to w pamięci to samo bo jedno jest miękką kopią drugiego zaś koszyk3 jest w pamięci oddzielnym blokiem więc nie jest taki sam

Zadanie 5:

$p \vee q \vee r \Rightarrow \neg p \Rightarrow (q \vee r) \wedge \neg p$

p	q	r	¬p	p ∨ q ∨ r	⇒ ¬p	(q ∨ r)	⇒ (q ∨ r)	∧ ¬p
0	0	0	1	0	1	0	0	0
0	0	1	1	1	1	1	1	1
0	1	0	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1
1	0	0	0	1	0	0	1	0
1	0	1	0	1	0	1	1	0
1	1	1	0	1	0	1	1	0

$p \Rightarrow \neg p \vee q$

p	q	¬p	p ⇒ ¬p	∨ q
0	0	1	1	1
0	1	1	1	1
1	0	0	0	0
1	1	0	0	1

$((p \Rightarrow q) \Rightarrow q \Rightarrow r) \Rightarrow (r \Rightarrow p) \Rightarrow q \Rightarrow p$

p	q	r	(p ⇒ q)	⇒ q	⇒ r	(r ⇒ p)	⇒ (r ⇒ p)	q	⇒ q	p	⇒ p
0	0	0	1	0	1	1	1	0	0	0	1
0	0	1	1	0	1	0	0	0	1	0	0
0	1	0	1	1	0	1	1	1	1	0	0
0	1	1	1	1	1	0	0	1	1	0	0
1	0	0	0	1	0	1	1	0	0	1	1
1	0	1	0	1	1	1	1	0	0	1	1
1	1	1	1	1	1	1	1	1	1	1	1

Wnioski:

Operatory logiczne pozwalają na przeprowadzanie skomplikowanych badań prawdziwosci zdania a w programach python pozwalają na przeprowadzanie analizy prawdziowosci podanych przez uzytkownika danych.